Raspberry Pi and C Programming

BARBARA HOHENSEE

Copyright © 2016 Barbara Hohensee

All rights reserved.

ISBN: 1536821128
ISBN-13: 978-1536821123

Contents

0 Preword.....5

1 Getting off the ground.....7

2 GETTING THE SOURCE CODE.....14

3 reviewing the hello program.....22

4 keyboard and screen.....24

5 Simple mathematical operations.....26

6 variables.....28

7 Arrays.....32

8 Conditions.....34

9 loops.....36

10 pointers.....37

11 COMMAND LINE.....39

12 Files.....40

13 Bitwise operators and system commands.....42

14 System commands.....44

15 Blinking led.....46

16 flowing led lights.....48

17 controlling a led by a button.....51

18 buzzer.....54

19 tilt sensor....58

20 rgb led....61

21 breathing led light....66

22 DHT11 – humidity sensor....69

23 use 74hc595 to control 7-segment....74

24 ultrasonic....80

25 dot-matrix....84

27 reed switch....89

28 relay....92

29 use 74hc595 to control led's....94

30 LCD display 1602....97

31 how to drive a dc motor....100

links....104

0 PREWORD

The objective of the book is to help you to understand how all your Raspberry Pi projects can be programmed using the C Language.

The first part gives you the basics of the C language. For this part, every Raspberry Pi on the market can be used.

The second part is about physical projects. The book contains 20 projects from blinking a LED to Tic Tac Toe and Bicycle Alarm. The Code for the second part assumes that you have a Raspberry Pi with 40 GPIO pins, like A+, B+, 2 or 3.

Recommendations:

- The latest Raspbian installed on an 8gb sd-card or bigger.
- For the source code editor, the recommendation is nano, which is used in this book and pre-installed on Raspbian. But others will work too.
- The Markdown editor used for this book is Remarkable.
- The project images are made with Fritzing.

Material for the physical projects in part 2 of the book:

- Your favorite Raspberry Pi
- Breadboard x 1
- Jumper cables, female to male x 20
- LED x 8, RGB LED x 1
- Resistor: 220 Ohm (10x), 1k Ohm (1x), 10k Ohm (1x)
- Button x 2
- Buzzer x 1
- DHT11 x 1
- NPN 8050 x 1
- Tilt sensor x 1
- Shift register 74HC595 x 2
- Displays: 7 segment display, Dot-matrix, LCD display 1602
- Photoresister x 1
- Diode 1N4007 x 1
- Relay x 1
- DC motor x 1
- Servo motor x 1
- Potentiometer x1

1 GETTING OFF THE GROUND

C is an excellent language for building fast, efficient and reliable programs. It can be found in applications where speed and memory footprint matter, such as the Linux kernel and data acquisition systems. But also embedded projects like alarm systems and many others can have advantages from the C Language.

3 things are needed:

1.C Compiler

2.Editor

3.C Library to get access to the GPIO pins

1. C Compiler

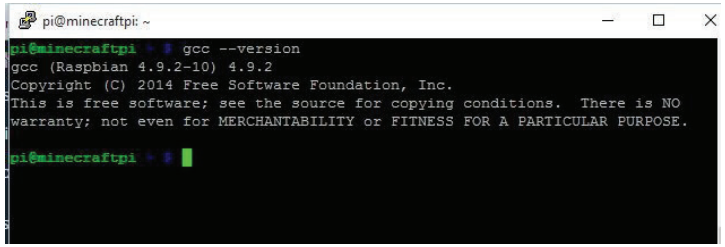
I assume that you have Raspbian installed. In this case, the "gcc" C Compiler is already pre-installed and is ready to use.

Testing the C Compiler:

Open a terminal window, like the LXTerminal

Bild

Write "gcc –version" to see a respond from the C Compiler. As a result, you should see the version of the Compiler.



2. Editor

Raspbian has already pre-installed several editors that can be used.

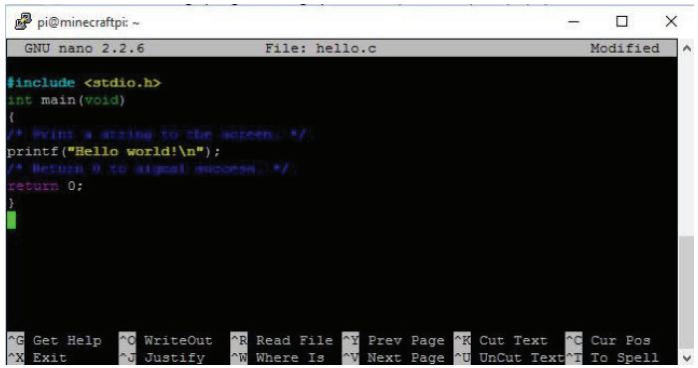
On the command line, you have the editor “nano” at your fingertips.

Nano – Editor example

Open a new or existing file called *hello.c*:

nano hello.c

Edit the content



Write the file to the disk:

<CTRL> + <O>

Exit the editor:

<CTRL> + <X>

Compiling

Compiling the file *hello.c* to an executable called *hello*:

gcc -o hello hello.c

Running the program:

./hello

3. C Library to get access to the GPIO pins

To be able to write a program in C for a physical project like the “Blinking LED”, you need to install one of the C libraries that have been written to access the GPIO pins of the Raspberry Pi.

The most common library used for this job is “wiringPi”.

WiringPi is a Wiring library written in C and should be usable from C, C++ and many other languages with suitable wrappers.
If you have ever used an Arduino before, you will know they are composed of two things. One is the hardware platform, and the other is the software platform. Part of the software side of things is a tool called Wiring. Wiring is the core of the input and output for the Arduino system.

Pin numbering

WiringPi supports both an Arduino style pin numbering scheme as well as the Raspberry Pi’s native BCM GPIO pin numbering scheme.

wiringPi Pin	BCM GPIO	Name	Header	Name	BCM GPIO	wiringPi Pin
–	–	3.3v	1 | 2	5v	–	–
8	R1:0/R2:2	SDA0	3 | 4	5v	–	–
9	R1:1/R2:3	SCL0	5 | 6	0V	–	–
7	4	GPIO7	7 | 8	TXD	14	15
–	–	0V	9 | 10	RXD	15	16
0	17	GPIO0	11 | 12	GPIO1	18	1
2	R1:21/R2:27	GPIO2	13 | 14	0V	–	–
3	22	GPIO3	15 | 16	GPIO4	23	4
–	–	3.3v	17 | 18	GPIO5	24	5
12	10	MOSI	19 | 20	0V	–	–
13	9	MISO	21 | 22	GPIO6	25	6
14	11	SCLK	23 | 24	CE0	8	10
–	–	0V	25 | 26	CE1	7	11
30	0	SDA. 0	27 | 28	SCL. 0	1	31
21	5	GPIO. 21	29 | 30	0V	–	–
22	6	GPIO. 22	31 | 32	GPIO. 26	12	26
23	13	GPIO. 23	33 | 34	0V	–	–
24	19	GPIO. 24	35 | 36	GPIO. 27	16	27
25	26	GPIO. 25	37 | 38	GPIO. 28	20	28
		0V	39 | 40	GPIO. 29	21	29
wiringPi Pin	BCM GPIO	Name	Header	Name	BCM GPIO	wiringPi Pin

For RPi B

For RPi B+/ 2 model B

Installing wiringPi

Step 1: Get the source code of wiringPi

```
git clone git://git.drogon.net/wiringPi
```

Step 2: Install wiringPi

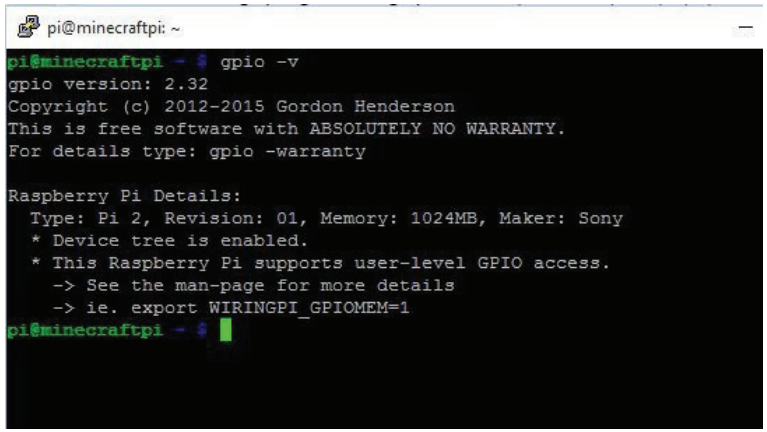
```
cd wiringPi

git pull origin

./build
```


Press Enter, with the script *build*, the source code of wiringPi will be compiled automatically and installed to the appropriate directory of Raspberry Pi OS.

Step 3: Test whether wiringPi is installed successfully or not

WiringPi includes many GPIO commands which enable you to control all kinds of interfaces on Raspberry Pi. You can test whether the wiringPi library is installed successfully or not by the following instructions.

gpio -v

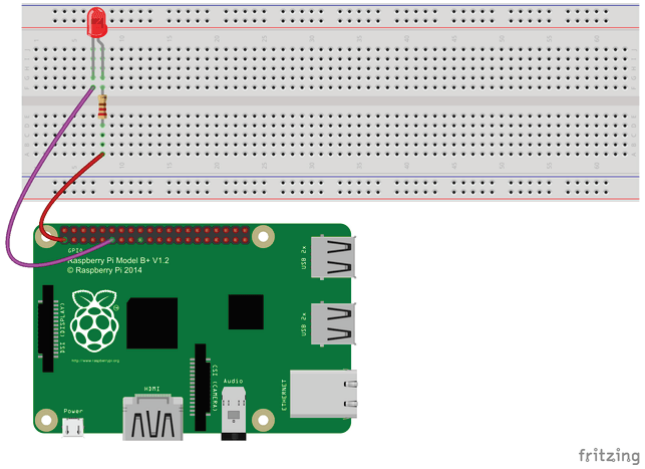


If the message above appears, the wiringPi is installed successfully.

gpio readall

```
pi@minecraftpi: ~
+-----+-----+---------+------+---+---Pi 2---+---+------+---------+-----+-----+
| BCM | wPi |   Name  | Mode | V | Physical | V | Mode | Name    | wPi | BCM |
+-----+-----+---------+------+---+----++----+---+------+---------+-----+-----+
|     |     |    3.3v |      |   |  1 || 2  |   |      | 5v      |     |     |
|   2 |   8 |   SDA.1 |   IN | 1 |  3 || 4  |   |      | 5V      |     |     |
|   3 |   9 |   SCL.1 |   IN | 1 |  5 || 6  |   |      | 0v      |     |     |
|   4 |   7 | GPIO. 7 |   IN | 1 |  7 || 8  | 1 | ALT0 | TxD     | 15  | 14  |
|     |     |      0v |      |   |  9 || 10 | 1 | ALT0 | RxD     | 16  | 15  |
|  17 |   0 | GPIO. 0 |   IN | 1 | 11 || 12 | 0 | IN   | GPIO. 1 | 1   | 18  |
|  27 |   2 | GPIO. 2 |   IN | 0 | 13 || 14 |   |      | 0v      |     |     |
|  22 |   3 | GPIO. 3 |   IN | 0 | 15 || 16 | 0 | IN   | GPIO. 4 | 4   | 23  |
|     |     |    3.3v |      |   | 17 || 18 | 0 | IN   | GPIO. 5 | 5   | 24  |
|  10 |  12 |    MOSI |   IN | 0 | 19 || 20 |   |      | 0v      |     |     |
|   9 |  13 |    MISO |   IN | 0 | 21 || 22 | 0 | IN   | GPIO. 6 | 6   | 25  |
|  11 |  14 |    SCLK |   IN | 0 | 23 || 24 | 1 | IN   | CE0     | 10  | 8   |
|     |     |      0v |      |   | 25 || 26 | 1 | IN   | CE1     | 11  | 7   |
|   0 |  30 |   SDA.0 |   IN | 1 | 27 || 28 | 1 | IN   | SCL.0   | 31  | 1   |
|   5 |  21 | GPIO.21 |   IN | 1 | 29 || 30 |   |      | 0v      |     |     |
|   6 |  22 | GPIO.22 |   IN | 1 | 31 || 32 | 0 | IN   | GPIO.26 | 26  | 12  |
|  13 |  23 | GPIO.23 |   IN | 0 | 33 || 34 |   |      | 0v      |     |     |
|  19 |  24 | GPIO.24 |   IN | 0 | 35 || 36 | 0 | IN   | GPIO.27 | 27  | 16  |
|  26 |  25 | GPIO.25 |   IN | 0 | 37 || 38 | 0 | IN   | GPIO.28 | 28  | 20  |
|     |     |      0v |      |   | 39 || 40 | 0 | IN   | GPIO.29 | 29  | 21  |
+-----+-----+---------+------+---+----++----+---+------+---------+-----+-----+
| BCM | wPi |   Name  | Mode | V | Physical | V | Mode | Name    | wPi | BCM |
+-----+-----+---------+------+---+---Pi 2---+---+------+---------+-----+-----+
pi@minecraftpi: ~ $
```

For the blinking LED example, first, connect circuit according to the following diagram:



Create the program with nano for example, call the file blink.c and include the wiringPi library.


```
pi@minecraftpi: ~/c-tut
  GNU nano 2.2.6                File: blink.c

#include <wiringPi.h>
#include <stdio.h>
#define LedPin 0

int main(void)
{
wiringPiSetup();

pinMode(LedPin, OUTPUT);

while(1){
digitalWrite(LedPin, LOW); //led on
printf("led on...\n");
delay(500);
digitalWrite(LedPin, HIGH); //led off
printf("...led off\n");
delay(500);
}
return 0;
}

                              [ Read 21 lines ]
^G Get Help  ^O WriteOut  ^R Read File ^Y Prev Page
^X Exit      ^J Justify   ^W Where Is  ^V Next Page
```

Save the file and leave the editor.

Compile

```
gcc -o blink blink.c -lwiringPi
```

Run the program as root (GPIO access needs it)

```
sudo ./blink
```

Stop the program wiith <CTRL> <C>

2 GETTING THE SOURCE CODE

The Source Code for all examples is on GitHub. You can ether download it or Fork it on GitHub to your own Repository.

1. Download the Source Code
Go to https://GitHub.com/eMakerStudio/C_for_Raspberry_Pi

and click on Download or Fork.

2. Connect yourself to GitHub
In case, you don't have an account on GitHub or don't know how to connect your Raspberry Pi to GitHub, follow this quick introduction.

Getting a GitHub Account
Go to https://GitHub.com/
Click on Sign up for GitHub and follow the instructions.

Connect your Raspberry Pi to GitHub

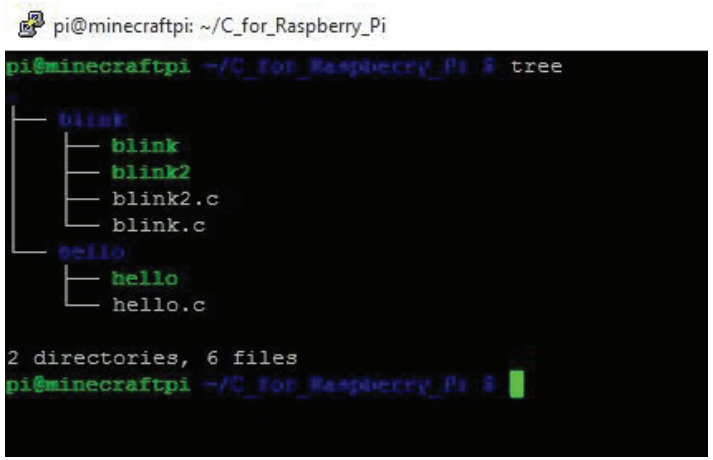
Step 1: Organize your local Code

Before syncing Source Code with GitHub, take a minute to organize your Code on the Raspberry.

Here is an example for the new Repository I created on GitHub for all the C Code for the Raspberry Pi.

So, I started with creating a directory on the Pi with the name "C_for_Raspberry_Pi".

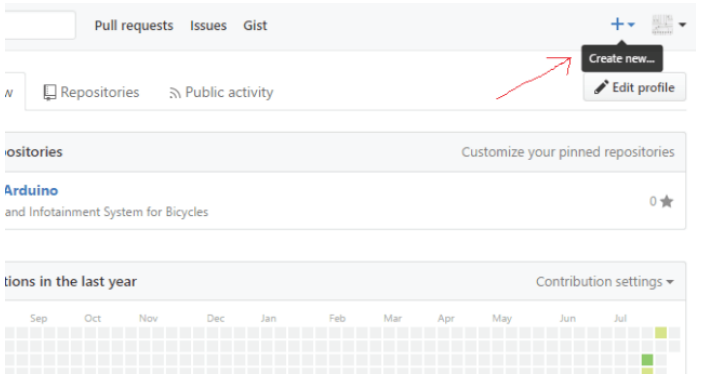
But it didn't felt right to put all Code in one directory. I think it's not a stupid idea that every project has its own folder, like this:



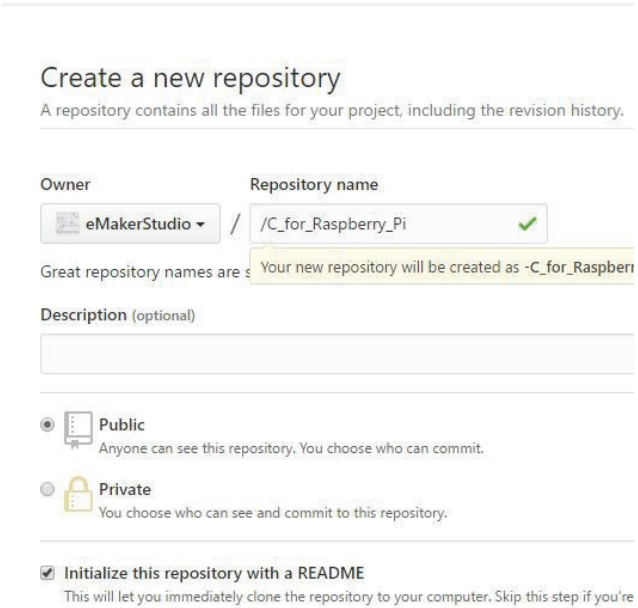
Now, let's get started with GitHub.

Step 2: Create a new Repository on GitHub

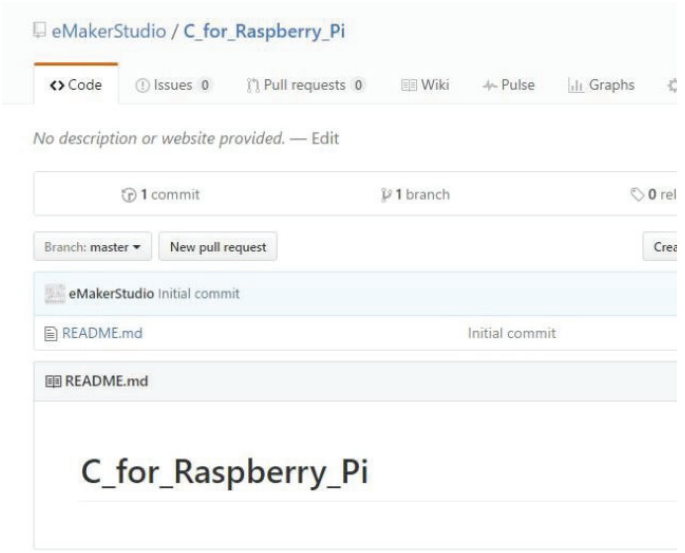
Login to your GitHub account and click on "create new" to create a new repository.



Fill in the name for your new repository and click on the green create button.



If you've clicked on "Initialize this repository with a README", GitHub will show this in your new repository.



Now, go back to the command line on the Raspberry Pi.

To work with GitHub from the Raspberry Pi, a program called "git" has to be installed. If it's not installed already, installed it with the command:

```
sudo apt-get install git
```

Step 3: Configuring git

The first thing you'll need to do is to specify your username and email, one at a time. These should match the ones on your GitHub account. In the Terminal window, type in:

```
git config --global user.name "username"
```

my "username" is eMakerStudio and my GitHub repositories are at https://GitHub.com/eMakerStudio — yours will be on your GitHub URL path.

Next, specify your email (which remains private). Enter your email like that:

```
git config --global user.email "your_email@example.com"
```

Step 4: Go to your local Source Code folder

In my case, I have to change to the folder C_for_Raspberry_Pi.

```
cd C_for_Raspberry_Pi/
```

Step 5: Initialize git

```
git init
```

Step 6: Pull the README from GitHub to your local repository

```
git remote add origin https://GitHub.com/<username>/<repo name>.git
```

In my case this would be:

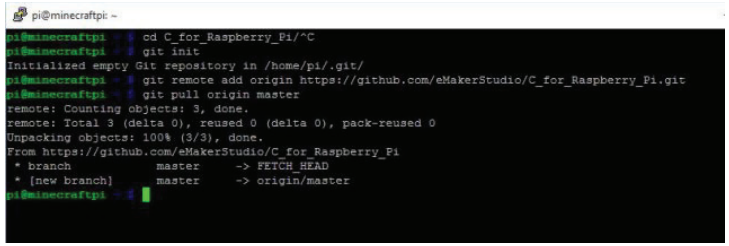
```
git remote add origin https://GitHub.com/eMakerStudio/C_for_Raspberry_Pi.git
```

To communicate with the outside world, Git uses *remotes*. These are repositories other than the one on your local disk which you can push your changes into (so that other people can see them) or pull from (so that you can get changes from others). This is essentially how this system works for Git and GitHub.

The command *git remote add origin* creates a new remote called *origin* located on GitHub.

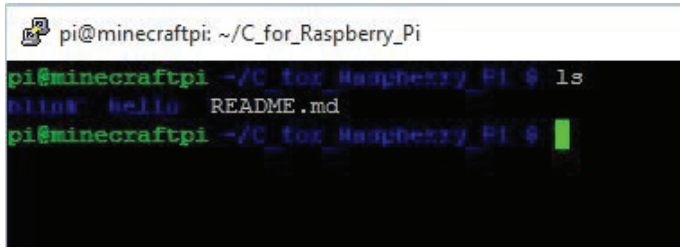
Once you do this, you can push to *origin* instead of type out the whole URL.

```
git pull origin master
```

This command will download the contents of your newly-created repository into your local directory. In this case, it is just the README.md file.

Control with the command ls that you got the README file and to make sure that everything is working properly.



The next step is to sync the local files with the remote GitHub repository.

Step 7: Add your local git files

```
git add .
```

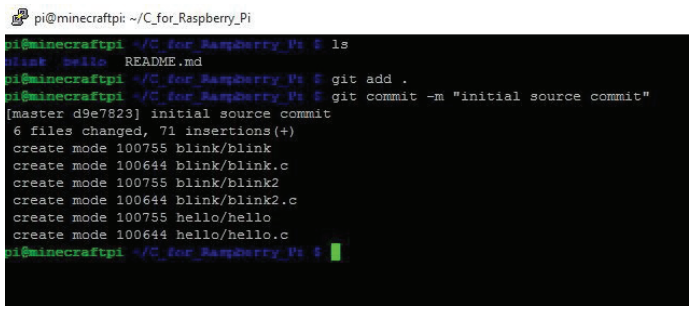
This will add all the files inclusive recursive directories to what will be a new "commit" of your source code.

Step 8: Commit the add

```
git commit -m "initial source commit"
```

The -m flag is used to add a message about your the commit. This should be a description of the most recent changes you've made.

You'll see some details, which shows which files were included in the commit.

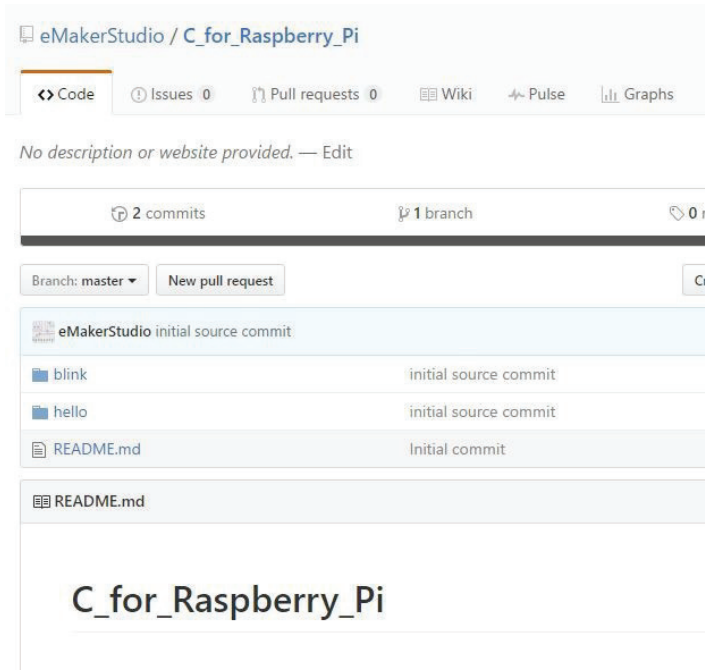


Step 9: Push the files to GitHub

```
git push origin master
```

This will upload your local repository into the remote GitHub repository that we previously called *origin,* and *master* is the branch name.

Your GitHub repository and your local repository have now the same content. Check it out.



After every change in your local repository (new Source Code files, new folders, ..), repeat the steps from above.

3 REVIEWING THE HELLO PROGRAM

Looking back to the program *hello.c* from the first chapter, we can in general say 2 things:

```
#include <stdio.h>
int main(void)
{
   /* Print a string to the screen. */
   printf("In the beginning... \n");
   /* Return 0 to signal success. */
   return 0;
}
```

1. The program is printing a string (visible)
2. The program will return an information to Linux (invisible)

To start with the very beginning of the program, we take a look at the import statement

#include <stdio.h>

The "stdio.h" library gives access to INPUT and OUTPUT channels of the system. By printing the text on the screen we need the OUTPUT functionality of the library.

The "stdio.h" library has a lot of inbuilt function. To learn more about it, write in a LXTerminal window:

man stdio

The next statement in our *hello.c* program

int main(void)

The execution of the program *hello.c* start from the "int main(void)" function.
The function has an integer (int) return value, which is given to the left of the function name. Any arguments passed into the function would go

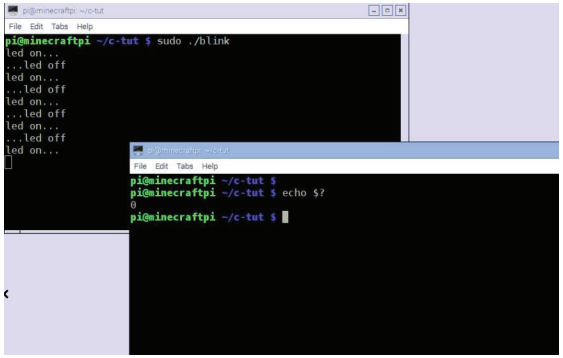
within the parentheses ().
In this case, the argument void has passed into the main function. This means that this function doesn't return anything.
The body of the function is defined by the brackets {}, which is referred to as a compound statement. A semicolon is used to complete each line or statement within the compound statement.
In the **hello.c** program there are only two statements:

- the printf function call to print a string to the screen,
- the return statement.

The text before the printf function and the return statement are comments. Comments can be added in two ways:
A one line comment stays behind //comment text.
A multiline comment stays between /* comment text */.

The difference between int main() and int main(void) is:
The main() function is deemed to take an unknown number of arguments. Using the keyword void within the brackets is the correct way to tell the compiler that the function takes NO arguments.

The "return 0" statement returns zero to the operating system. This is not visible on the screen. But we can control what a program has been passed to the operating system because the value of the return statement is stored in the BASH shell variable. Type in a second terminal window echo $? to print the value of $? on the screen.



4 KEYBOARD AND SCREEN

There are many standard library functions defined in the stdio.h header file. Some of these functions can be used to read from the keyboard and write to the screen.
Input values can be read from the keyboard using the scanf function.

Writing the Code, for example with nano.
nano keyboard.c

```
#include <stdio.h>
int main()
{
  /* Variable declaration to hold the input from the keyboard */
  int age;
  /* Ask the user for input */
  printf("How old are you? ");
  /* Read the number that comes from the keyboad input */
  scanf("%d", &age);
  /* Print the users input on the screen */
  printf("You are %d years old. \n", age);
  return 0;
}
```

Compiling
gcc -o keyboard keyboard.c

Saving
<CTRL> <o>

Leaving the editor
<CTRL> <x>

Running
./keyboard

Execution

```
pi@minecraftpi: ~/c-tut
pi@minecraftpi ~/c-tut $ ./keyboard
How old are you? 24
You are 24 years old,
pi@minecraftpi ~/c-tut $
```

In the keyboard.c program an integer variable is declared to hold a value. The integer variable corresponds to a space in the memory of the

computer, which is allocated to hold the value. The user is as asked their age. Then the program waits for the input from the keyboard. The input is read from the keyboard using the scanf function. The character cod %d tells scanf to read an integer from the keyboard. The integer is written into the memory allocated to the age variable by passing the memory address of the age variable, which is accessed via &age. The age value is then written back to the screen using printf. Notice that printf also uses the %d character code to denote integer output.

To read more about scanf, write into a terminal window:
man scanf

If everything is working fine, you may want to keep you GitHub Repository up-to-date, repeat step 7 and forward from the previous chapter.

```
pi@minecraftpi: ~/C_for_Raspberry_Pi
pi@minecraftpi ~/C_for_Raspberry_Pi $ git add .
pi@minecraftpi ~/C_for_Raspberry_Pi $ git commit -m "initial source commit"
[master 33bef70] initial source commit
 2 files changed, 15 insertions(+)
 create mode 100755 keyboard/keyboard
 create mode 100644 keyboard/keyboard.c
pi@minecraftpi ~/C_for_Raspberry_Pi $ git push origin master
Username for 'https://github.com': eMakerStudio
Password for 'https://eMakerStudio@github.com':
Counting objects: 5, done.
Delta compression using up to 4 threads.
Compressing objects: 100% (5/5), done.
Writing objects: 100% (5/5), 3.04 KiB | 0 bytes/s, done.
Total 5 (delta 1), reused 0 (delta 0)
To https://github.com/eMakerStudio/C_for_Raspberry_Pi.git
   d9e7823..33bef70  master -> master
pi@minecraftpi ~/C_for_Raspberry_Pi $
```

5 SIMPLE MATHEMATICAL OPERATIONS

Computers are able to perform many mathematical calculations quickly, even the oldest Raspberry Pi.
A first example.

```
 GNU nano 2.2.6                    File: simple_math.c

#include <stdio.h>
int main()
{
  // Addition
  /* Declare three integer variables */
  int x, y, z;
  printf("enter two whole numbers, separated by a space: ");
  /* Read two values from the keyboard */
  scanf("%d %d",&x,&y);
  /* add the two values together and pass the result to z */
  z = x + y;
  printf("%d + %d = %d\n", x,y,z);

  // Multiplication
  /* multiply x by 10 and then assign the result to x */
  x = x * 10;
  printf("The result of x is: %d\n ", x);
  return 0;
}
```

The program simple_math.c demonstrates addition (+) and multiplication (*) operators. In a similar manner, substraction (-) and divistion (/) can also be used. Mathematical expressions within a program are executed sequentially. For example, x=x*10 will start from the current value of x, multiply it by ten and assign the result to x.

Execution

```
pi@minecraftpi ~/C_for_Raspberry_Pi/simple_math $ ./simple_math
enter two whole numbers, separated by a space: 2 3
2 + 3 = 5
The result of x is: 20
 pi@minecraftpi ~/C_for_Raspberry_Pi/simple_math $
```

What do you expect from the next program for result?

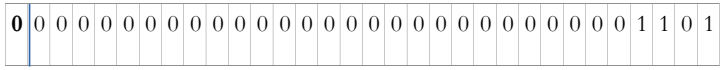

```c
#include <stdio.h>
int main()
{
  int i = 100, j = 9, k;
  i = i/10;
  k = i -j;
  printf("%d - %d = %d\n" ,i,j,k);
  return 0;
}
```

6 VARIABLES

There are two categories of numeric variables in C:

- whole number, declared as int or double
- floating point numbers declared as float

When a variable is declared, a block of memory is allocated to the variable. The amount of memory allocated to an int/ double and float is the same, but the way the number is stored is different. And int is stored in memory as a sign it and a 31bit value. For example,



is 13 in decimal.
From the right to the left, the bits correspond to 2^0, 2^1, …. 2^30. The sign bit is on the far left. The number thirteen is formed from the sum of the bits. The largest number which can be stored in and int is 2^31-1, which is in decimal 2147483647. If one more is added to this number, the number becomes -2147483647. This happens because the sign bit is set.

Here is a simple test program (variable_int_test.c):

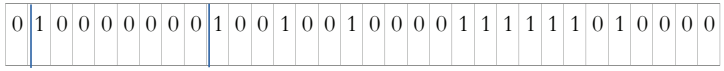
```
 GNU nano 2.2.6                File: variable_int_test.c

#include <stdio.h>
int main()
{
  /* Assigning (2^31)-1 to i */
  int i = 2147483647;
  i++; // increment by 1
  printf("%d\n",i); // print the value of i
  return 0; // return success to the OS
}
```

The result of the execution:

```
pi@minecraftpi ~/C_for_Raspberry_Pi/variable_int_test $ ./variable_int_test
-2147483648
pi@minecraftpi ~/C_for_Raspberry_Pi/variable_int_test $ echo $?
0
pi@minecraftpi ~/C_for_Raspberry_Pi/variable_int_test $
```

An int cannot be used to store a fraction of a number.

Fractional or floating-point numbers can be stored in a float. Unlike an int, a float hat three pieces: a sign, an exponent and a mantissa.
For example,



is 3.14159 in decimal.
The sign bit is on the far left, the exponent bits are in the middle and the mantissa bits are on the far right. The number is formed by the multiplication:

sign x 2^exponent x mantissa

The value of the exponent is stored in the same way as the int. However, the meaning of the mantissa bits is different:
from left to right, the bits correspond to 2^0, 2^-1, 2^-2, etc.. If a floating-point number becomes too big, it will become not-a-number or infinity. If the precision bits in the mantissa are all used additional precision will be lost. This loss of precision is called a floating-point error.
In the following example (floating_point_error.c), the addition of 0.00000001 to the float f is lost.

```
 GNU nano 2.2.6              File: floating_point_error.c

#include <stdio.h>
int main()
{
  /* Assigning the approximate value of PI. */
  float f = 3.14159;
  /* Add a very small number to it */
  f+=0.0000001;
  /* Print the value stored in f */
  printf("%f\n",f);
  /* Return success to the OS */
  return 0;
}
```

Going back to some simple mathematics.
The division of float and int variables behaves differently. An integer division will always be rounded down to the nearest int, whereas a float will be rounded to the precision of the mantissa.
Example (division.c)

```
 GNU nano 2.2.6                File: division.c

#include <stdio.h>
int main()
{
  /* Declare an int and assign it the value three. */
  int i = 3;
  /* Declare a float and assign it the value three. */
  float f = 3;
  /* Divide i by two and assign the result to i. */
  i /= 2;
  /* Divide f by two and assign the result to f. */
  f /= 2.;
  /* Print the result */
  printf("int (3/2) = %d, float (3./2.) = %f\n",i,f);
  /* Return success to the OS. */
  return 0;
}
```

The result of this program is,

```
int (3/2) = 1, float (3./2.) = 1.500000
```

Characters are stored in a similar way as an int.
A char is a signed 8-bit number. It can be used to print the associated ASCII character or the position of the ASCII character withing the table.

Example

```
 GNU nano 2.2.6                File: characters.c

#include <stdio.h>
int main()
{
  /* Declare a character c and assign it 'a' */
  char c = 'a';
  /* Print the value */
  printf("The character \'%c\' has the ASCII value %d\n" ,c,c);
  /* Return success to the operating system. */
  return 0;
}
```

The program's output:

```
The character 'a' has the ASCII value 97
```

7 ARRAYS

A string is represented in memory as a series of sequential characters. A series of sequential values of the same type is called an array.

An array of ten integers can be declared by,

```
int array[10];
```

Each element in an array can be accessed using the corresponding index. Similar to many other languages, these indexes start from zero and have up to N-1 where N is the size of the array.
When an array is declared, a sequential block of memory is allocated. The value in each array index may or may not be zero. Therefore, it is important to initialize each array element with a value. Arrays may be declared and assigned values at the same time,

```
int array[10] = {1,2,3,4,5,6,7,8,9,10}
```

or each element of the array can be initialized individually.

Example (array.c)

```
 GNU nano 2.2.6                File: array.c

#include <stdio.h>
int main()
{
  /* Create an array with four elements and assign values. */
  float fourVector[4] = {1.0, 2.0, 0.5, 2.292};
  /* Print the number of elements in the array. */
  printf("There are %ld elements\n" , sizeof(fourVector)/sizeof(float));
  /* Print the fourth element. */
  printf("fourVector[3]=%f\n", fourVector[3]);
  /* Return the success to the operation system */
  return 0;
}
```

In this example, the sizeof command returns the size of the array as a whole. Therefor, dividing this by sizeof(float) returns the number of elements in the array.

A string of nine characters can be stored in a character array of ten characters,

```
char str[10];
```

The tenth element is used to store the string terminating character '\0'. An array of strings can be stored as an array of an array of characters.
Here is a simple string example (string.c),

```
 GNU nano 2.2.6                File: string.c

#include <stdio.h>
int main()
{
  /* Create a 50 character string */
  char str[50] = "A test string";
  /* Print the string */
  printf("%s\n" ,str);
  printf("The third character = " "%c\n" ,str[2]);
  return 0;
}
```

The outcome from this program:

```
A test string
The third character = t
```


8 CONDITIONS

Many different conditions can be expressed in the C language. These conditions can be used within if-else statements, switch, or within loops. A summery of logic conditions is provided in the table below,

Condition	Meaning
a == b	a 'equal' b
a != b	a 'not equal' b
a > b	a 'greater than' b
a <= b	a 'less than or equal' b
!a	'not' a
a || b	a 'or' b
a && b	a 'and' b

These conditions can be combined together to produce more complicated conditions. When several conditions are combined together it is important to use parentheses, to set the order in which the conditions are tested. For example,
(a || b) && c is not the same as **a||(b && c)**, since the logic conditions within the parentheses is evaluated first.

Conditions can be used within if else statements to choose a particular outcome,

Example (conditions.c)

```
 GNU nano 2.2.6              File: conditions.c

#include <stdio.h>
int main()
{
  /* Start with twenty apples. */
  int apples = 20;
  /* Set the price */
  float cost, costPerApple = 0.20;
  /* Ask the customer */
  printf("How many apples would you like to buy?\n");
  /* Read the number input from the keyboard */
  scanf("%d",&apples);
  /* Check, if there are enough apples left */
  if(apples > 20) {
   /* Apologise */
    printf("Sorry, we only have 20 apples left.\n");
  }
  else {
    /* Calculate the total cost. */
    cost = costPerApple*apples;
    /* Ask for the money */
    printf("That will be %.2f BatCoins please.\n" ,cost);
  }
  return 0;
}
```


9 LOOPS

Repetitive operations, such as accessing array elements or performing the same mathematical operation several times can be more efficiently implemented using loops. C offers three main loop types: for, while, and do while.

Example with the for loop (loops.c),

```
 GNU nano 2.2.6                File: loops.c

#include <stdio.h>
int main()
{
  int i;
  /* Loop from zero to nine */
  for(i=0; i<10; i++) {
     /* Print all value of i */
     printf(" i = %d\n" ,i);
  }
  return 0;
}
```

Between the parentheses of the for loop are three sections: (initialization; condition; incrementation). The initialization happens once at the start of the loop. Then the loop continues until the condition is false. Each loop executes the compound statement within the {} brackets and then executes the incrementation arguments.

10 POINTERS

Unlike simple variables, a pointer stores the address of a memory location. The memory location can be a simple variable, a function, or a structure of some sort. The type of the pointer normally has to be the same as the type of the variable, to which the pointer points. When implemented correctly within a program, using pointers can produce more efficient code. A pointer is declared by prepending a variable name with an asterix. For example, a pointer of type int can be declared via,

```
int *p = 0;  /* Declare a pointer p and assign it a null address. */
```

The pointer can then be given the address of another variable,

```
int i; /* Declare an int I. */
p = &i; /* Assign the address of I to p. */
```

The assignment can also take place through a function call,

```
 GNU nano 2.2.6                File: pointers.c

#include <stdio.h>
/* A function with no return value. */
void fun(int, int *);
int main()
{
  /* Initialise two int variables */
  int np = 1, p = 1;
  /* Print the address */
  printf("&np=%p, p=%p\n" , &np, &p);
  /* Print the values */
  printf("Before fun(): np=%d, p=%d\n" , np, p);
  /* Pass the value of np and the address of p. */
  fun(np, &p);
  /* Return success to the operating system */
  return 0;
}

void fun(int np, int *p)
{
  /* Assign 2 to the local variable np. */
  np = 2;
  /* Assign 2 to the memory of p defined in main. */
  *p = 2;
  /* Print the address. */
  printf("&np=%p, p=%p\n" , &np, p);
}
```


In this example, the address of the variable p defined in the main function is passed to the pointer p in the function fun. The value in this memory address is then changed by dereferencing the pointer, by using an asterix in front of the pointer name. Notice that when a pointer is declared an asterix must be used. However, when a pointer is declared, the pointer can be assigned an address rather than a value.

Pointers can also be used in arrays.

11 COMMAND LINE

Arguments from the command line can be utilized by using another form of the main function,

```
 GNU nano 2.2.6            File: command_line.c

#include <stdio.h>
int main(int argc, char *argv[]) {
  int i;
  for(i=0; i<argc; i++) {
    printf("argv[%d] = %s\n" ,i,argv[i]);
  }
  return 0;
}
```

The arguments are passed as an array of strings, where a string is a character array. Test this program out by running it with or without some command line arguments, for example,

```
./command_line
./command_line firstArg secondArg
```


12 FILES

Other than reading from the keyboard or writing to the screen, it is often necessary to write to or read from files stored on a hard drive or memory card.
File accesses can be made using binary or ACSII mode. In this tutorial only ASCII mode will be discussed. Before a file can be opened a FILE pointer should be declared,

/* Declare a file pointer and set it to null */
FILE **filePtr = 0;**

Then the file can be opened for reading "r", or writing "w".
An example, how to write to a file called textFile.txt.

/* Open textFile.txt for writing */
FilePtr = fopen("textFile.txt", "w");

If the fopen call fails, the a FILE pointer is set to null. Once the file has been opened, commands very similar to those used to rad from the keyboard or write to the screen can be used to access the file,

```
fprint(filePtr, "Writing out some important data .. %d, %d, %d\n" ,2,3,4);
```

When the file access has been completed it has to be closed, flushing any data in memory to the disk,

```
fclose(filePtr); /*Close textFile.txt, flushing data to disk */
```

The final example program (file.c) uses many of the concepts discussed so far to provide a simple encryption algorithm,

```
#include <stdio.h>
int main(int argc, char *argv[]) {
  int mask = 163;
  char c;
  FILE *inputFile = 0, *outputFile = 0;

  if(argc!=3) {
    printf(" Usage: %s <input file> <output file>\n" ,argv[0]);
    return 1;
  }

  inputFile = fopen(argv[1], "r");
  if(!inputFile) return 2;

  outputFile = fopen(argv[2], "w");
  if(!outputFile) return 3;

  c = fgetc(inputFile);
  while(c != EOF) {
    c ^= mask;
    fputc(c,outputFile);
    c = fgetc(inputFile);
  }

  fclose(inputFile);
  fclose(outputFile);

  return 0;
}
```

The program uses an exclusive-OR to encrypt and decrypt files.
An exclusive-OR is true if either bit is set, ad false if both or no bits are set.
For example, 2 ^3 = 1 in decimal or in binary 10 ^11 = 01.
To test the program, type in a text file with nano and then try to encode the file. Then run the program again to decode the file.

13 BITWISE OPERATORS AND SYSTEM COMMANDS

The main bitwise operators are summarized in the table below.

Condition	Meaning
a & b	a 'and' b
a | b	a 'or' b
a ^ b	a 'exclusive or' b
a >> n	right shift a by n
a << n	left shift a by n

These operators are typically used with integer variable types or signal bytes stored in char variables. They act on the binary form of the number and are typically used for bit packing or testing packed bits.
For example, if the status of several switches needs to be read, their input could be stored in one integer variable.

Example (bitwise.c)

```
 GNU nano 2.2.6                File: bitwise.c

#include <stdio.h>
int main() {
  int bit = 0, i = 1;
  while(i>0) {
    printf(" pow(2, %2d) = %11d\n" ,bit,i);
    i = i<<1;
    bit++;
  }
  return 0;
}
```

In this example, the value stored in the variable i is shifted one place to the left. The left shift operator has the effect of moving all of the bits in the variable i one place to the left. If a bit is shifted outside the memory allocation of the variable I, the bit is lost. In this case, I only contains one.

Therefore, the action of the left shift operator is to move to causes the while loop to stop.

14 SYSTEM COMMANDS

It can be useful to be able to run shell commands or other programs without directly linking to an associated library.
This can be accomplished with the system function,

Example (system.c)

```
 GNU nano 2.2.6                File: system.c

#include <stdio.h>
int main() {
  /* List the files in current directory */
  system("ls ./");
  return 0;
}
```

The system function evaluates the string argument passed to it as if it had been typed at the command line. The standard output from the command is not captured by the program and is instead printed on the screen.

The standard output from a system command or program can be captured using a pipe. Pipes follow the same syntax as regular file functions, allowing reading, write and bidirectional connections.
For examples, the contents of the current directory can be read into a file using (pipes.c),

```
 GNU nano 2.2.6                File: pipes.c

#include <stdio.h>
int main() {
  int c;
  /* Create a null FILE pointer */
  FILE *ptr = 0 ;
  /* List the files in a directory and listen */
  ptr = popen("ls ./" , "r");
  /* If the command fails return failure  */
  if(!ptr) return 1;
  /* Read each character */
  while((c = fgetc(ptr)) != EOF) {
    /* Print the characters */
    printf("%c" ,(char)c);
  }
  /* Close the pipe */
  pclose(ptr);
  return 0;
}
```

In this case, each file name returned is available within the program.

Any command that can be typed at the command line can be executed using system or popen.

Another example (monitoring.c) shows how to use a function like sysinfo to monitor our Linux system,

```
  GNU nano 2.2.6                File: monitoring.c

#include <stdio.h>
#include <sys/sysinfo.h>

int main() {
  /* Create a sysinfo instance to hold the result. */
  struct sysinfo info;
  /* Get the system information */
  sysinfo(&info);
  printf("Memory used = %d\n" ,info.totalram - info.freeram);
  return 0;
}
```

Before the system information can be retrieved, struct variable of sysinfo type is created. This is not a simple variable, but contains several variables. The member variables of the struct are accessed using "." operator. When sysinfo is called, the address of the struct variable of sysinfo type is passed to the function. The function then writes the status into the member variables of the struct.

This is the End of part 1. There is more to learn about the C language.
But the lessons we got so far should you help to understand the basic concepts and get you started with embedded projects.

15 BLINKING LED

Introduction

In this lesson, we will learn how to program your Raspberry Pi to make an LED blink. You can play numerous tricks with an LED as long as your imagination is rich enough. Now follow me to learn, and you will enjoy the fun of DIY at once.

Needed material

- 1*Breadboard
- 1*LED
- 1*Resistor (220Ω)
- Jumper wires

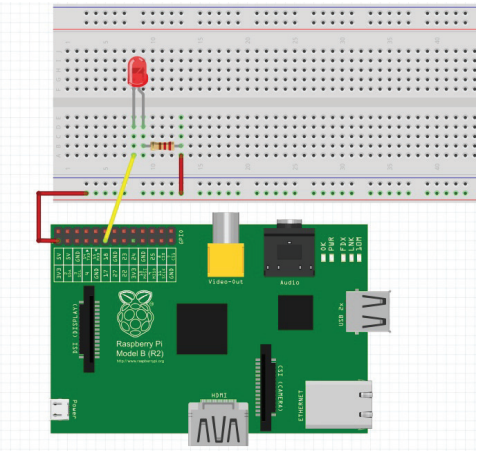
Experimental Principle

Semiconductor light-emitting diode is a type of component which can turn electric energy into light energy via PN junctions. According to its wavelength, semiconductor light-emitting diode can be categorized into laser diode, infrared light-emitting diode and visible light-emitting diode which, called light-emitting diode for short, is usually known as LED.
When we supply 2V-3V forward voltage to an LED, it will blink if only forward currents flow through the LED. Usually we have red, yellow, green, blue and color-changing LED which can change its colors with different voltages. LEDs are widely used due to its low operating voltage, low current, luminescent stability and small size.
LEDs are diodes too. Hence they have a voltage drop which usually varies between 1V-3V depending on their types. Likewise, LEDs usually can emit light if supplied 5mA-30mA current, and generally we use 10mA-20mA. So when an LED is used, it is necessary to connect a current-limiting resistor to protect the LED from over-burning.

In this experiment, we will connect a 220Ω resistor to the positive pole of the LED and then connect it to 3.3 V power source, and connect the negative pole of LED to GPIO0 (See the Raspberry Pi pins distribution diagram and the above picture). If we write 1 to GPIO0, the voltage of the pin is 3.3V and the LED will not blink; if we write 0 to GIIO0, the output voltage is 0, then the LED will light up according to the above principle.

Experimental Procedure

Step 1:
Connect the circuit as shown in the following diagram



Step 2:
Edit and save the code with for example nano and call the file blink.c.

Step 3:
Compile the code

```
gcc -o blink blink.c -lwiringPi
```

Step 4:
Run the program

```
./blink
```

Now, you should see your LED blinking. Cancel the program with <CTRL> +<C>

Further Exploration
If you want the LED to blink faster, just change the delay time. For example, change the time to delay (200), recompile the program, and then run the program, you will see the LED blinking faster.

16 FLOWING LED LIGHTS

Introduction

In this lesson, we will learn how to make eight LED's blink in various effects as you want based on Raspberry Pi.

Experimental Conditions

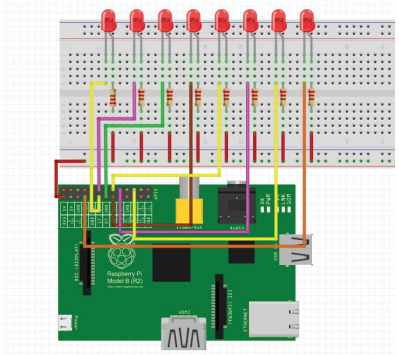
- 1*Breadboard
- 8*LED
- 8*Resistor (220Ω)
- Jumper wires

Experimental Principle

Set GPIO0 to GPIO7 to low in turn by programming, then LED0 to LED7 will light up in turn. You can make eight LED's blink in different effects by controlling their delay time and the order of lighting up.

Experimental Procedures

Step 1:
Connect the circuit as shown in the following diagram



Step 2:
Edit and save the code in a file called 8Led.c

Step 3:
Compile the code

```
gcc -o 8Led 8Led.c  -lwiringPi
```

Step 4:
Run the program

```
 ./8Led
```

Now, when you press Enter, eight LED's will light up circularly and render different effects.

Further Exploration

We can write the blinking effects of LED's in an array. If you want to use one of these effects, you can call it in the **main()** function directly.

8Led.c

```
#include <wiringPi.h>
#include <stdio.h>

// led_n on
void led_on(int n)
{
	digitalWrite(n, LOW);
}

// led_n off
void led_off(int n)
{
	digitalWrite(n, HIGH);
}

int main(void)
{
	int i;

	if(wiringPiSetup() == -1){
		printf("setup wiringPi failed !");
		return 1;
	}

	for(i=0;i<8;i++){
		printf("linker LedPin : GPIO %d(wiringPi pin)\n",i); //when initialize wiring successfully,print message to screen
	}

	for(i=0;i<8;i++){	//make 8 pins' mode is output
		pinMode(i, OUTPUT);
	}

	while(1){
		for(i=0;i<8;i++){	//make led on from left to right
			led_on(i);
			delay(100);
			led_off(i);
		}
	//	delay(500);
		for(i=8;i>=0;i--){	//make led off from right to left
			led_on(i);
			delay(100);
			led_off(i);
		}
	}
```

17 CONTROLLING A LED BY A BUTTON

Introduction

In this lesson, we will learn how to turn an LED on or off by a button.

Experimental Conditions

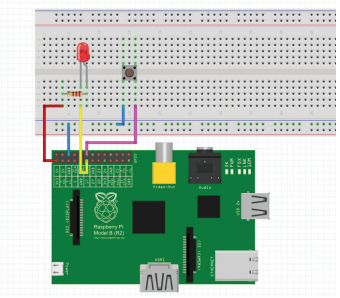
- 1*Breadboard
- 1*LED
- 1*Button
- 1*Resistor (220Ω)
- Jumper wires

Experimental Principle

Use a normally open button as an input of Raspberry Pi, when the button is pressed, the GPIO (General Purpose Input/Output) connected to the button will turn into low level (0V). We can detect the state of the GPIO connected to the button through programming. That is, if the GPIO turns into low level, it means the button is pressed, you can run the corresponding code according to this condition. In this experiment, we make the LED light up.

Experimental Procedures

Step 1: Connect the circuit as shown in the following diagram



Step 2:

Edit and save the code in a file called BtnAndLed.c

Step 3: Compile the code

```
gcc -o BtnAndLed BtnAndLed.c -lwiringPi
```

Step 4:

Run the program

```
./BtnAndLed
```

Press Enter, when you press the button, the LED will light up; when you release the button, the LED will go out.

Summary

Through this experiment, you have basically mastered the Input and Output programming operation of Raspberry Pi GPIOs. I hope you can make persistent efforts and continue to learn the next contents.

BtnAndLed.c

```c
#include <wiringPi.h>
#include <stdio.h>

#define LedPin    0
#define ButtonPin 1

int main(void)
{   /* if initialize wiring failed, print message to screen */
        if(wiringPiSetup() == -1){
                printf("setup wiringPi failed !");
                return 1;
        }

        pinMode(LedPin, OUTPUT);
        pinMode(ButtonPin, INPUT);

        /* pull up to 3.3V or 5V, gives GPIO1 more stability */
        pullUpDnControl(ButtonPin, PUD_UP);
        while(1){
                digitalWrite(LedPin, HIGH);
                /* indicate that the button has pressed */
                if(digitalRead(ButtonPin) == 0){
                        digitalWrite(LedPin, LOW);   //led on
                }
        }

        return 0;
}
```


18 BUZZER

Introduction

In this lesson, we will learn how to drive an active buzzer with a PNP transistor to make sounds.

Experimental Conditions

- 1*Breadboard
- 1*Buzzer (Active)
- 1*NPN transistor (8050)
- 1*Resistor (1KΩ)
- Jumper wires

Experimental Principle

As a type of electronic buzzer with integrated structure, buzzers, which use DC power supply, are widely used in computers, printers, photocopiers, alarms, electronic toys, automotive electronic equipments, telephones, timers and other electronic products for voice devices. Buzzers can be categorized as piezoelectric and magnetic buzzers. A piezoelectric buzzer is mainly composed of multivibrator, piezoelectric buzzer slice, impedance matcher, resonance chamber, shell, etc. A magnetic buzzer is mainly composed of oscillator, electromagnetic coil, magnet, vibrating diaphragm, shell, etc. Buzzers can also be categorized as active and passive buzzers (See the following pictures). When we place the pins of two buzzers upwards, we can see the one with green circuit board is a passive buzzer, while the one without circuit board instead of enclosing with black tape is an active buzzer.

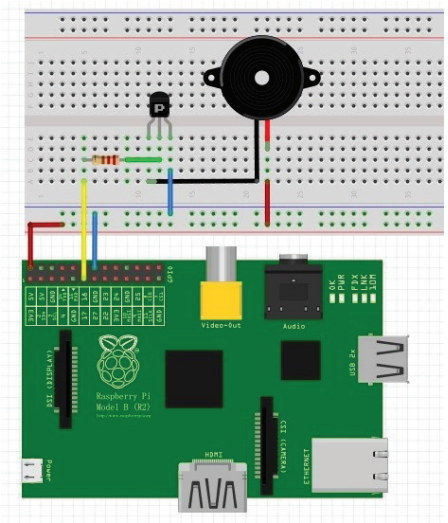
The difference between an active buzzer and a passive buzzer is:

The active buzzer has built-in oscillating source, so it will make sounds as long as it is electrified. While the passive buzzer does not have oscillating source, so it will not tweet if you use DC signals,

instead you must use square waves whose frequencies are between 2K and 5K to drive it. The active buzzer is often more expensive than the passive because multiple built-in oscillating circuits exist.
In this experiment, we use an active buzzer. When we make the GPIO of Raspberry Pi output low level (0V) by programming, the transistor will conduct because of current saturation and the buzzer will make sounds. But when we supply high level to the IO of Raspberry Pi, the transistor will cut-off and the buzzer will not make sounds.

Experimental Procedures

Step 1:
Connect the circuit as shown in the following diagram (Pay attention to the positive and negative poles of the buzzer)



Step 2:
Edit and save the code in a file called beep.c

Step 3:
Compile the code

```
gcc -o beep beep.c -lwiringPi
```

Step 4:
Run the program

```
./beep
```

Now, you should hear the buzzer make sounds.

Further Exploration

If you have a passive buzzer in hand, you can replace the active buzzer with it. So you can make it sound “do re mi fa so la si do” through programming as long as you have some programming foundation and enough patience.

beep.c

```
#include <wiringPi.h>
#include <stdio.h>

#define BeepPin 0

int main(void)
{
        if(wiringPiSetup() == -1){ //when initialize wiring failed,print messageto screen
                printf("setup wiringPi failed !");
                return 1;
        }

        pinMode(BeepPin, OUTPUT);   //set GPIO0 output

        while(1){
                digitalWrite(BeepPin, LOW);  //beep on
                delay(100);                  //delay
                digitalWrite(BeepPin, HIGH); //beep off
                delay(100);                  //delay
        }

        return 0;
}
```


19 TILT SENSOR

Introduction

The tilt-switch we use here is a ball tilt-switch with a metal ball inside. It is used to detect small angle of inclination.

Components

- 1*Tilt-switch
- 1*Resistor (220Ω)
- 1*LED
- Several jumper wires

Experimental Principle

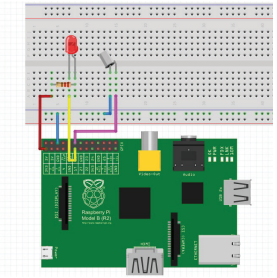
The principle is very simple. It mainly uses the ball in the switch changing with different angle of inclination to achieve the purpose of triggering circuits. When the ball in tilt switch runs from one end to the other end because of external force shaking, the tilt switch will conduct, or it will break.

In this experiment, we will use an LED and a tilt-switch to build a tilt indication light. When the tilt-switch is tilt, the LED will light up. Otherwise, it will go out.

Experimental Procedures

Step 1:

Connect the circuit



Step 2:
Edit and save the code in a file called tiltSwitch.c

Step 3:
Compile the code
```
gcc -o tiltSwitch tiltSwitch.c -lwiringPi
```

Step 4:
Run the program
```
./tiltSwitch
```

Press Enter, if you tilt the switch, the LED will light up. Otherwise, it will go out.

tiltSwitch.c

```c
#include <stdio.h>
#include <string.h>
#include <errno.h>
#include <stdlib.h>
#include <wiringPi.h>

#define  LedPin     0
#define  TiltPin    1

int main(void)
{
        if(wiringPiSetup() < 0){
                fprintf(stderr, "Unable to setup wiringPi:%s\n",strerror(errno));
                return 1;
        }

        pinMode(TiltPin, INPUT);
        pinMode(LedPin, OUTPUT);
        pullUpDnControl(TiltPin, PUD_UP);

        while(1){
                if(0 == digitalRead(TiltPin)){
                        digitalWrite(LedPin, LOW);
                }
                else{
                        digitalWrite(LedPin, HIGH);
                }
        }

        return 0;
}
```

20 RGB LED

Introduction

RGB LED's can emit various colors of light. They are manufactured by packaging three LED's of red, green, and blue into a transparent or semitransparent plastic shell and lead out four pins. The three primary colors of red, green, and blue can be mixed to all kinds of colors by brightness, so you can make RGB LED's emit light with all kinds of colors by controlling the circuit.

Components

- 1*RGB LED
- 3*Resistor (220Ω)
- Several jumper wires

Experimental Principle

The three primary color red, green and blue of a RGB LED can compose various colors by brightness. We can adjust the brightness of LED with PWM technology. Raspberry Pi has only one channel hardware PWM output, but we need three channels to control the RGB LED. As a result, it is difficult to realize with the hardware PWM of Raspberry Pi. Do not worry! Fortunately the softPwm library simulates PWM (softPwm) for us with software method. Based on this, we only need to include the header file softPwm.h, then call the API it provided to easily achieve multi-channel PWM output to control the RGB LED to display all kinds of colors.

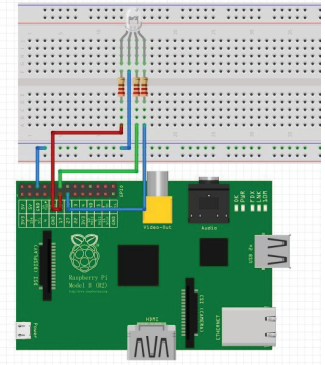
There are two types of package for RGB LED's. One is patch type, and the other is dual-in-line type. The difference between them is color resolution. The former has better color resolution.

RGB LED's can be categorized into common anode type and common cathode type. In this experiment, we use common cathode RGB LED.

Experimental Procedures

Step 1:

Connect the circuit as shown in the following diagram



Step 2:

Edit and save the code in a file called rgb.c

Step 3:

Compile the code

```
gcc -o rgb.c -lwiringPi -lpthread
```

Step 4:

Run the program

```
./rgb
```

Press Enter, you will see the RGB LED light up, and display different colors in turn.

Further Exploration

I hope you can modify the parameters of the function ledColorSet( ) by yourself, then compile and run the program to see the color changes of the RGB LED.

Experimental Summary

You have learnt how to control RGB LEDs with softPwm of the Raspberry Pi through this lesson. I hope you can continue to explore softPwm application in DC motor speed regulation.

rgb.c

```
#include <wiringPi.h>
#include <softPwm.h>
#include <stdio.h>

#define uchar unsigned char

#define LedPinRed    0
#define LedPinGreen  1
#define LedPinBlue   2

void ledInit(void)
{
        softPwmCreate(LedPinRed,  0, 100);
        softPwmCreate(LedPinGreen,0, 100);
        softPwmCreate(LedPinBlue, 0, 100);
}

void ledColorSet(uchar r_val, uchar g_val, uchar b_val)
{
        softPwmWrite(LedPinRed,   r_val);
        softPwmWrite(LedPinGreen, g_val);
        softPwmWrite(LedPinBlue,  b_val);
}

int main(void)
{
        int i;

    /* when initialize wiring failed, print message to screen */
        if(wiringPiSetup() == -1){
                printf("setup wiringPi failed !");
                return 1;
        }
        //printf("linker LedPin : GPIO %d(wiringPi pin)\n",LedPin);
```

```
        //when  initialize  wiring  successfully,print  message  to
screen

        ledInit();

        while(1){
                ledColorSet(0xff,0x00,0x00);   //red
                delay(500);
                ledColorSet(0x00,0xff,0x00);   //green
                delay(500);
                ledColorSet(0x00,0x00,0xff);   //blue
                delay(500);

                ledColorSet(0xff,0xff,0x00);   //yellow
                delay(500);
                ledColorSet(0xff,0x00,0xff);   //pick
                delay(500);
                ledColorSet(0xc0,0xff,0x3e);
                delay(500);

                ledColorSet(0x94,0x00,0xd3);
                delay(500);
                ledColorSet(0x76,0xee,0x00);
                delay(500);
                ledColorSet(0x00,0xc5,0xcd);
                delay(500);

        }

        return 0;
}
```


21 BREATHING LED LIGHT

Introduction

In this lesson, we will gradually increase and decrease the luminance of an LED with PWM (Pulse-width modulation) technology, which looks like breathing. So we give it a magical name - Breathing Light.

Experimental Conditions

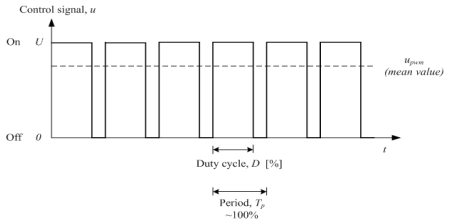
- 1*Breadboard
- 1*LED
- 1*Resistor (220Ω)
- Jumper wires

Experimental Principle

Before we talk about PWM, let's have a look at the applications of PWM first. PWM has been successfully applied in motor speed regulation, steering angle control, light intensity control and signal output. For example, when PWM is applied to a horn, it will make a sound. After we know about its special functions, let's find out what PWM really is.

Pulse Width Modulation commonly refers to PWM. Pulse Width Modulation (PWM) is a digital coding method for analog signal levels. Since a computer cannot output an analog voltage but digital voltage value 0V or 3.3V, we modulate the duty cycle of square waves to encode a specific level of analog signal by using a high-resolution counter. PWM signals are essentially digital signals, for the full amplitude DC power supply is either 3.3V (ON) or 0V (OFF) at any given time. Voltage or current source is applied to an analog load in the form of ON or OFF repetitive pulse sequence. When it is on, DC power supply will be applied to the load; when it is off, DC power supply will be disconnected. If only the bandwidth is wide enough, any analog value can be encoded by PWM. The output voltage value is calculated by the on and off time, $V_{out} = (T_{on} / T)*V_{max}$.

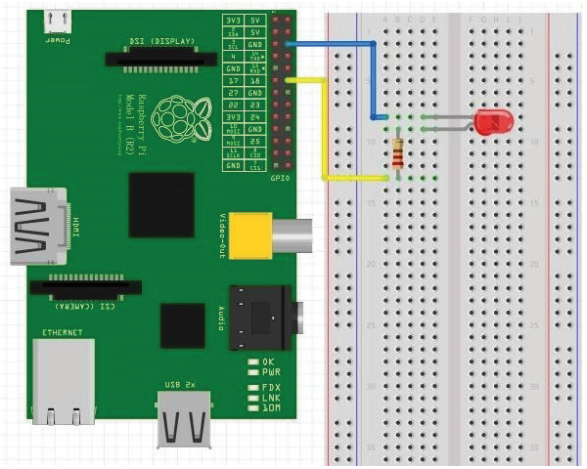
Here is the introduction to three basic parameters of PWM:



1.The term duty cycle describes the proportion of 'on' time to the regular interval or 'period' of time
2.The term period describes the reciprocal of pulses in one second
3.Voltage amplitude (e.g. 0V-3.3V)

Experimental Procedures

Step 1:
Connect the circuit as shown in the following diagram



Step 2:

Edit and save the code in a file called breathLed.c

Step 3:
Compile the code

```
gcc -o breathLed brathLed.c -lwiringPi
```

Step 4:
Run the program

```
./breathLed
```

Press Enter and you will see a gradual change of the LED luminance, rendering the effect of breathing.

Summary

Through this experiment, you should have mastered the technical principle of PWM and how to program Raspberry Pi with PWM. You can apply this technology to DC motor speed regulation in the future.

22 DHT11 – HUMIDITY SENSOR

Introduction

The digital temperature and humidity sensor DHT11 is a composite sensor that contains a calibrated digital signal output of temperature and humidity. The technology of a dedicated digital modules collection and the temperature and humidity sensing technology are applied to ensure that the product has high reliability and excellent long-term stability.

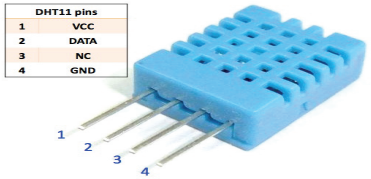
The sensor includes a resistive sense of wet component and an NTC temperature measurement device, and is connected with a high-performance 8-bit microcontroller.

Components

- 1 * Breadboard
- 1 * Humidity sensor
- 1 * 3-Pin anti-reverse cable

Experimental Principle

Only three pins are available for use: VCC, GND, and DATA.

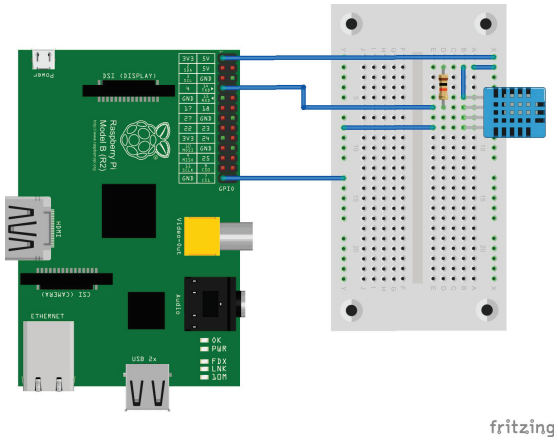


The communication process begins with the DATA line sending start signal to DHT11, and DHT11 receives the signal and returns an answer signal, then the host receives the answer signal and begins to receive 40-bit humidity data (8-bit humidity integer + 8-bit humidity decimal + 8-bit temperature integer + 8-bit temperature decimal + 8-bit checksum). For more information, please refer to the datasheet of DHT11.

Experimental Procedures

Step 1:

Build the circuit



Raspberry Pi	Humidity sensor
GPIO0	SIG
5V	VCC
GND	GND

Step 2:

Edit and save the Code in a file called dht11.c

Step 3:

Compile

```
gcc -o dht dht.c -lwiringPi
```

Step 4:

Run

```
sudo ./dht
```

dht11.c

```
#include <wiringPi.h>
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>

#define MAXTIMINGS 85

#define DHTPIN 0

int dht11_dat[5] = {0,0,0,0,0};

void read_dht11_dat()
{
        uint8_t laststate = HIGH;
        uint8_t counter = 0;
        uint8_t j = 0, i;
        float f; // fahrenheit

        dht11_dat[0] = dht11_dat[1] = dht11_dat[2] = dht11_dat[3] =
dht11_dat[4] = 0;

        // pull pin down for 18 milliseconds
        pinMode(DHTPIN, OUTPUT);
        digitalWrite(DHTPIN, LOW);
        delay(18);
        // then pull it up for 40 microseconds
        digitalWrite(DHTPIN, HIGH);
        delayMicroseconds(40);
        // prepare to read the pin
        pinMode(DHTPIN, INPUT);

        // detect change and read data
        for ( i=0; i< MAXTIMINGS; i++) {
                counter = 0;
                while (digitalRead(DHTPIN) == laststate) {
                        counter++;
                        delayMicroseconds(1);
                        if (counter == 255) {
                                break;
                        }
                }
                laststate = digitalRead(DHTPIN);

                if (counter == 255) break;

                // ignore first 3 transitions
                if ((i >= 4) && (i%2 == 0)) {
                        // shove each bit into the storage bytes
                        dht11_dat[j/8] <<= 1;
                        if (counter > 16)
                                dht11_dat[j/8] |= 1;
                        j++;
                }
        }

        // check we read 40 bits (8bit x 5 ) + verify checksum in
the last byte
        // print it out if data is good
        if ((j >= 40) &&
```

```
                        (dht11_dat[4]     ==     ((dht11_dat[0]     +
dht11_dat[1] + dht11_dat[2] + dht11_dat[3]) & 0xFF)) ) {
                f = dht11_dat[2] * 9. / 5. + 32;
                printf("Humidity = %d.%d %% Temperature = %d.%d *C
(%.1f *F)\n",
                                dht11_dat[0],           dht11_dat[1],
dht11_dat[2], dht11_dat[3], f);
        }
        else
        {
                printf("Data not good, skip\n");
        }
}

int main (void)
{

        printf  ("Raspberry  Pi  wiringPi  DHT11  Temperature  test
program\n") ;

        if (wiringPiSetup () == -1)
                exit (1) ;

        while (1)
        {
                read_dht11_dat();
                delay(1000); // wait 1sec to refresh
        }

        return 0 ;
}
```


23 USE 74HC595 TO CONTROL 7-SEGMENT

Introduction

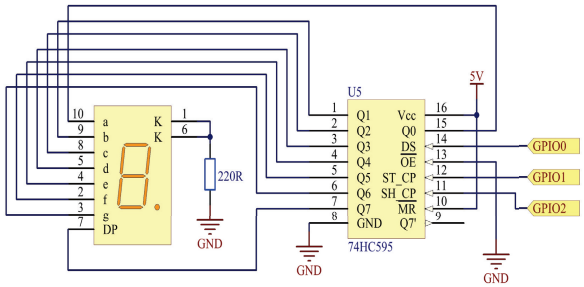
In this lesson, we will learn how to use 74HC595 to drive a 7-segment display to cycle a figure from 0 to 9.

Components

- 1* Breadboard
- 1* 74HC595
- 1* 7-segment display
- 1* Resistor (1KΩ)
- Jumper wires

Principle

A 7-segment display is an 8-shaped component which packages many LED's. Its leads are connected internally. External pins are its segments and common electrodes. The segments are respectively labeled by letter a, b, c, d, e, f, g, and dp. 7-segment displays can be categorized into two types: common cathode and common anode, depending on different light-emitting diode connections. They are widely used in electronic appliances especially home appliances such as air conditioning's, water heaters, refrigerators and so on, because of its low price, ease of use, high brightness and long service life. Most temperature display units of water heater are 7-segment displays.

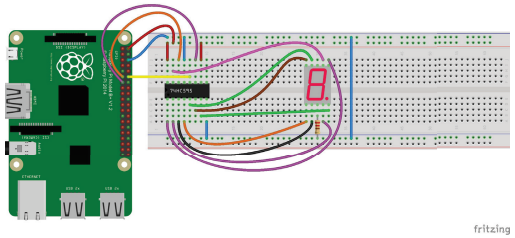


In this experiment, a common cathode 7-segment display is used, which connects all the cathodes of LED's together to form a common cathode (COM) electrode. It should be connected to ground. When the anode of an LED in a certain segment is at high level, the corresponding segment will light up; when it is at low, the segment will not light up.

We connect pin ST_CP of 74HC595 to Raspberry Pi GPIO1, SH_CP to GPIO2, DS to GPIO0, parallel output ports to 8 segments of the LED segment display. Input data in DS pin to shift register when SH_CP (the clock input of shift register) is at the rising edge and to memory register when ST_CP (the clock input of memory) is at the rising edge. As a result, we can control the states of SH_CP and ST_CP via the Raspberry Pi GPIO's to transform serial data input into parallel data output so as to save Raspberry Pi GPIO's and to drive the LED segment display.

Experimental Procedures

Step 1:
Build the circuit



Step 2:
Edit and save the Code in a file called segment1.c

Step 3:
Compile


```
gcc segment1.c -o segment1 -lwiringPi
```

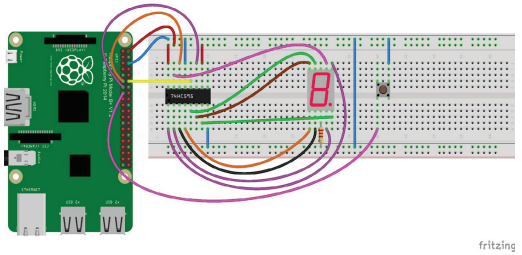
Step 4:
Run

```
sudo ./segment1
```

You should see the 7-segment display cycle from 0 to 9, and A to F.

Further Exploration

You can slightly modify the hardware and software based on this experiment to make a dice. For hardware, you add a button to the original board. Then the modified circuit is as follows:



The example Code is called `dice.c`

Run

```
sudo ./dice
```

You should see numbers between 0 and 6 flashing quickly on the segment display. Press the button on the breadboard, and the segment display will statically display a random number between 0 and 6 for 2 seconds and then circularly display random numbers between 0 and 6 quickly again.

Summary

Through this lesson, you have mastered the basic principle and programming for a 7-segment display based on Raspberry Pi as well as more knowledge about using 74HC595. You can apply what you've learnt and put it into practice.

segment1.c

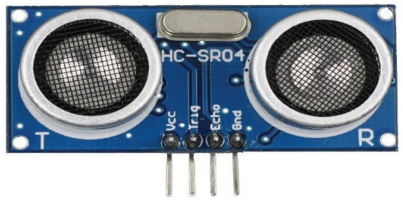
```
#include <wiringPi.h>
#include <stdio.h>

#define   SDI   0   //serial data input
#define   RCLK  1   //memory clock input(STCP)
#define   SRCLK 2   //shift register clock input(SHCP)

unsigned              char              SegCode[17]              =
{0x3f,0x06,0x5b,0x4f,0x66,0x6d,0x7d,0x07,0x7f,0x6f,0x77,0x7c,0x39,0
x5e,0x79,0x71,0x80};

void init(void)
{
        pinMode(SDI, OUTPUT); //make P0 output
        pinMode(RCLK, OUTPUT); //make P0 output
        pinMode(SRCLK, OUTPUT); //make P0 output

        digitalWrite(SDI, 0);
        digitalWrite(RCLK, 0);
        digitalWrite(SRCLK, 0);
}

void hc595_shift(unsigned char dat)
{
        int i;

        for(i=0;i<8;i++){
                digitalWrite(SDI, 0x80 & (dat << i));
                digitalWrite(SRCLK, 1);
                delay(1);
                digitalWrite(SRCLK, 0);
        }

                digitalWrite(RCLK, 1);
                delay(1);
```

```
                digitalWrite(RCLK, 0);
}

int main(void)
{
        int i;

        if(wiringPiSetup()   ==   -1){   //when   initialize   wiring
failed,print messageto screen
                printf("setup wiringPi failed !");
                return 1;
        }

        init();

        while(1){
                for(i=0;i<17;i++){
                        hc595_shift(SegCode[i]);
                        delay(500);
                }
        }

        return 0;
}
```


24 ULTRASONIC

Introduction

The ultrasonic sensor uses sound to accurately detect objects and measure distances. It sends out ultrasonic waves and converts them into electronic signals.



Components

- 1 * Breadboard
- 1 * Ultrasonic ranging module
- Several jumper wires (Male to Female)
- 1 * Resistor 1k Ohm

Experimental Principle

This sensor works by sending a sound wave out and calculating the time it takes for the sound wave to get back to the ultrasonic sensor. By doing this, it can tell us how far away objects are relative to the ultrasonic sensor.

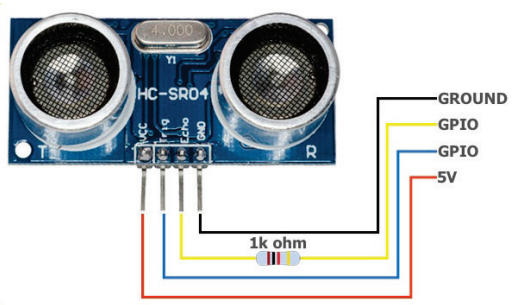
Test distance = (high level time * velocity of sound (340M/S)) / 2 (in meters)

Experimental Procedures

HC-SR04 ultrasonic distance sensor module has four pins:

VCC – 5V, input power
TRIG – Trigger Input
ECHO– Echo Output
GND– Ground

Step 1:
Build the circuit



Step 2:
Edit and save the Code in a file called ultrasonic.c

Step 3:
Compile
gcc -o ultrasonic ultrasonic.c -lwiringPi

Step 4: Run
sudo ./ultrasonic

Now you can see the distance between the ultrasonic ranging module and the obstacle in front on the screen. Sway your hand over the ultrasonic ranging module slowly and observe the distance printed on the screen.

C Code

```
#INCLUDE <WIRINGPI.H>
#INCLUDE <STDIO.H>
#INCLUDE <SYS/TIME.H>

#DEFINE TRIG    4
#DEFINE ECHO    5

VOID ULTRAINIT(VOID)
{
        PINMODE(ECHO, INPUT);
        PINMODE(TRIG, OUTPUT);
}

FLOAT DISMEASURE(VOID)
{
        STRUCT TIMEVAL TV1;
        STRUCT TIMEVAL TV2;
        LONG START, STOP;
    FLOAT DIS;

        DIGITALWRITE(TRIG, LOW);
        DELAYMICROSECONDS(2);

        DIGITALWRITE(TRIG, HIGH);
        DELAYMICROSECONDS(10);
        DIGITALWRITE(TRIG, LOW);

        WHILE(!(DIGITALREAD(ECHO) == 1));
        GETTIMEOFDAY(&TV1, NULL);

        WHILE(!(DIGITALREAD(ECHO) == 0));
        GETTIMEOFDAY(&TV2, NULL);

        START = TV1.TV_SEC * 1000000 + TV1.TV_USEC;
        STOP  = TV2.TV_SEC * 1000000 + TV2.TV_USEC;

        DIS = (FLOAT)(STOP - START) / 1000000 * 34000 / 2;
        //DIS = (STOP - START) * 34000 / 1000000 / 2;

        RETURN DIS;
}

INT MAIN(VOID)
{
        FLOAT DIS;

        IF(WIRINGPISETUP() == -1){ //WHEN INITIALIZE WIRING
FAILED,PRINT MESSAGETO SCREEN
                PRINTF("SETUP WIRINGPI FAILED !");
                RETURN 1;
        }

        ULTRAINIT();

        WHILE(1){
                DIS = DISMEASURE();
                PRINTF("DISTANCE = %0.2F CM\N",DIS);
                DELAY(1000);
        }
```

```
        RETURN 0;
}
```


25 DOT-MATRIX

Introduction

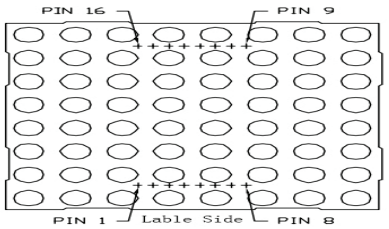
In this lesson, we will learn how to use 74HC595 to drive an LED dot-matrix.

Components

- 1* Breadboard
- 2* 74HC595
- 1* Dot-Matrix
- Jumper wires

Principle

The external view of a dot-matrix is shown as follow:



For pin definition:
Define row and column numbering at first (only for the dot-matrix ending with BS)

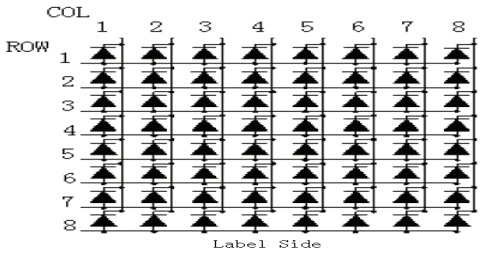
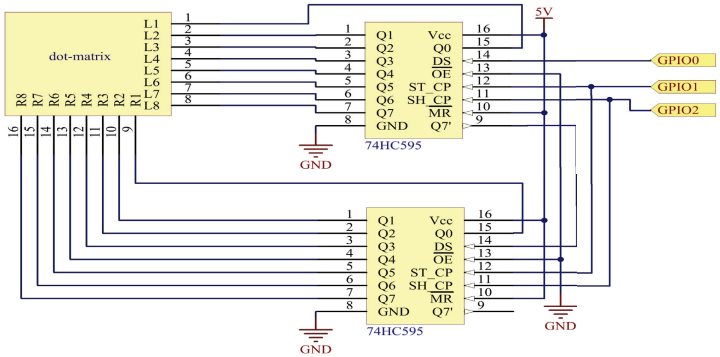


Table for pin number corresponding to rows and columns:

COL	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**
Pin No.	**13**	**3**	**4**	**10**	**6**	**11**	**15**	**16**
ROW	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**
Pin No.	**9**	**14**	**8**	**12**	**1**	**7**	**2**	**5**

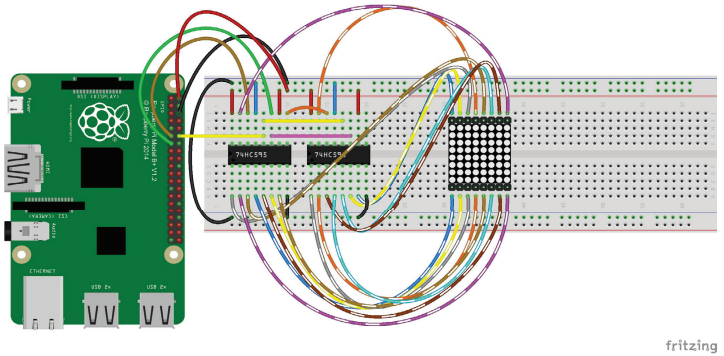
The display principle of the 8*8 dot-matrix:
The 8*8 dot-matrix is made up of sixty-four LED's and each LED is placed at the cross point of a row and a column. When the electrical level of a certain row is High and the electrical level of a certain column is Low, then the corresponding LED will light up; if you want to light the LED on the first dot, you should set ROW 1 to high level and COL 1 to low level, then the LED on the first dot will light up; if you want to light the LED's on the first row, you should set ROW 1 to high level and COL (1, 2, 3, 4, 5, 6, 7, 8) to low level, then all the LED's on the first row will light up; if you want to light the LED's on the first column, you should set COL 1 to low level and ROW (1, 2, 3, 4, 5, 6, 7, 8) to high level, then all the LEDs on the first column will light up.

The principle of 74HC595 has been illustrated previously. One chip is used to control the rows of the dot-matrix while the other, the columns.



Experimental Procedures

Step 1:
Build the circuit



Step 2:
Edit and save the Code in a file called dotMatrix.c

Step 3:
Compile

```
gcc dotMatrix.c -o dotMatrix -lwiringPi
```

Step 4:
Run

```
sudo ./dotMatrix
```

You should see LED's light up as you want.

Summary
Through this lesson, you have mastered the basic principle of LED dot-matrix and how to program Raspberry Pi to drive an LED dot-matrix based on 74HC595 cascade.

26 PHOTORESISTOR

Introduction
A photoresistor or light-dependent resistor (LDR) or photocell is a light-

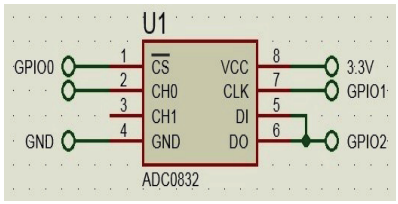
controlled variable resistor. The resistance of a photoresistor decreases with increasing incident light intensity; in other words, it exhibits photoconductivity. A photoresistor can be applied in light-sensitive detector circuits, and light- and dark-activated switching circuits.

Components
- 1*Resistor(10K)
- 1*ADC0832
- 1*Photoresistor
- Several jumper wires

Experimental Principle
With light intensity increasing, the resistance of a photoresistor will decrease. Thus the output voltage will change.

Experimental Procedures

Step 1:
Connect the circuit



Step 2:
Edit and save the code in a file called photoresistor.c

Step 3:
Compile the code

```
gcc -o photoresistor photoresistor.c -lwiringPi
```

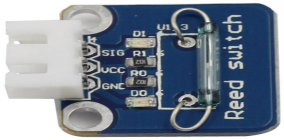
Step 4:
Run the program
sudo ./photoresistor

Now, if you change light intensity, you will find the value printed on the screen change accordingly.

27 REED SWITCH

Introduction

A reed switch (as shown below) is used to detect the magnetic field. Hall sensors are generally used to measure the speed of intelligent vehicles and count in assembly lines, while reed switches are often used to detect the existence of a magnetic field.



Components

- 1 * Breadboard
- 4 * Jumper wires (Male to Male, 2 red and 2 black)
- 1 * Reed switch module
- 1 * Dual-color LED module
- 2 * 3-Pin anti-reverse cable
- 1 * Magnet (Self provided)

Experimental Principle

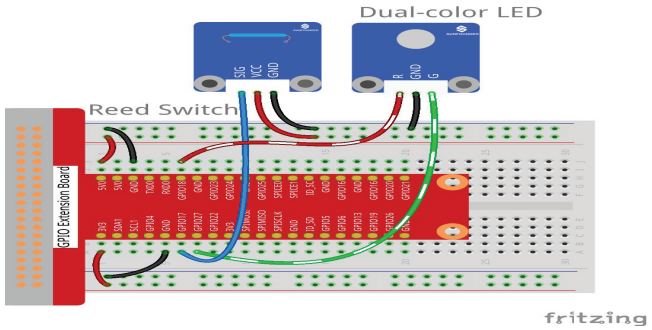
A reed switch is a type of line switch component that realizes control by magnetic signals. It induces by a magnet. The “switch" here means dry reed pipe, which is a kind of contact passive electronic switch component with the advantage of simple structure, small size, and convenient control. The shell of a reed switch is commonly a sealed glass pipe in which two iron elastic reed electroplates are equipped and inert gases are filled. Normally, the two reeds made of special materials in the glass tube are separated. However, when a magnetic substance approaches the glass tube, the two reeds in the glass tube are magnetized to attract each other and contact under the function of magnetic field lines. As a result, the two reeds will pull together to connect the circuit connected with the nodes.

After external magnetic force disappears, the two reeds will be separated with each other because they have the same magnetism, so the circuit is also disconnected. Therefore, as a line switch component controlling by magnetic signals, the dry reed pipe can be used as a sensor to count, limit positions and so on. At the same time, it is widely used in a variety of communication devices.

Experimental Procedures

Step 1:

Build the circuit



Raspberry Pi	T-Cobbler	Reed Switch
GPIO0	GPIO17	SIG
5V	5V0	VCC
GND	GND	GND

Raspberry Pi	T-Cobbler	Dual-color LED
GPIO1	GPIO18	R
GND	GND	GND
GPIO2	GPIO27	G

Step 2:

Edit and save the Code in a file called reed_switch.c

Step 3:

Compile

```
gcc -o reed_switch reed_switch.c -lwiringPi
```

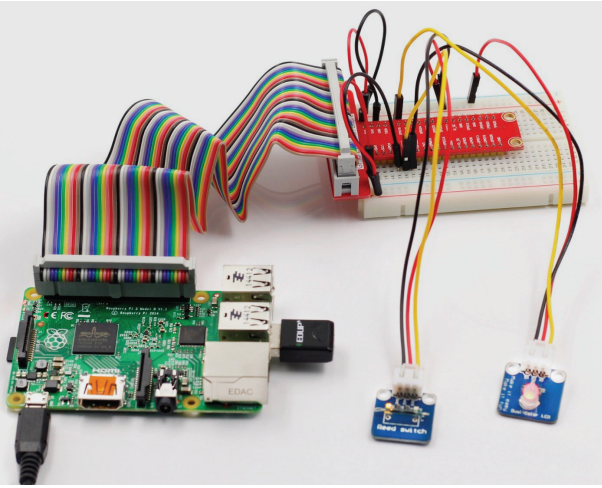
Step 4:

Run

```
sudo ./reed_switch
```

Then the LED will emit green light. Place a magnet near the reed switch, "Detected Magnetic Material!" will be printed on the screen

and the LED will emit red light. Move away the magnet, the LED will emit green light again.



28 RELAY

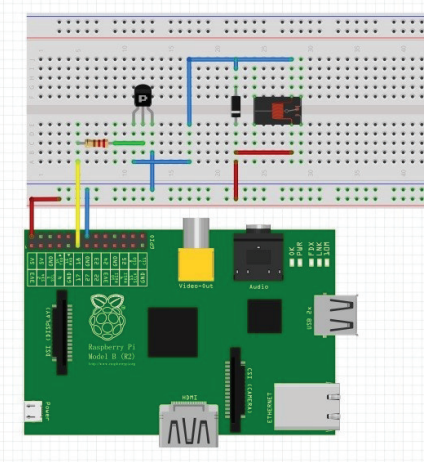
Introduction
Relays are suitable for driving high power electric equipment, such as lights, electric fans and air conditioning. We can use a relay to realize low voltage to control high voltage by connecting it to MCU.

Components
- 1*Diode (1N4007)
- 1*NPN transistor (8050)
- 1*Resistor(1KΩ)
- 1*Relay
- Several jumper wires

Experimental Principle
When we make the IO connected to the Raspberry Pi and the transistor outputs low level (0V) by programming, the transistor will conduct because of current saturation. The normally open contact of the relay will be closed, while the normally closed contact of the relay will be broken; when outputting high level (3.3V), the transistor will be cut off, and the relay will recover to initial state.

Experimental Procedures

Step 1:
Connect the circuit according to the following method



Step 2:
Edit and save the code in a file called relay.c

Step 3:
Compile the code

```
gcc -o relay relay.c -lwiringPi
```

Step 4:
Run the program

```
sudo ./relay
```

Now, press Enter, and you should be able to hear ticking sound. This sound is made by breaking normally closed contact and closing normally open contact. You can attach a high voltage device you want to control, for example, a bulb with 220V voltage, to the output port of relay, so the relay plays a role of automatic switch.

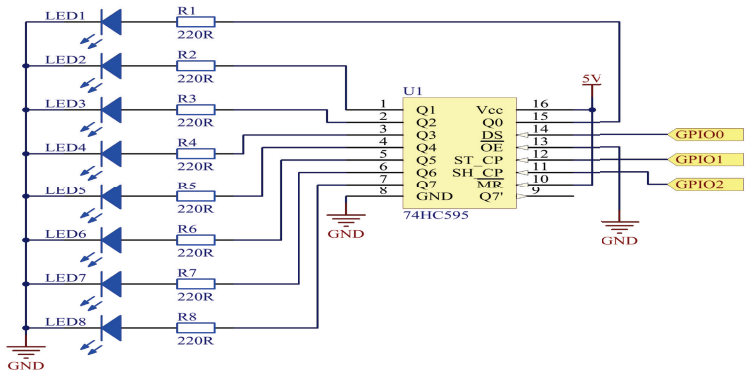
29 USE 74HC595 TO CONTROL LED'S

Introduction
In this lesson, we will learn how to use 74HC595 to make eight LED's blink regularly.

Components
- 1* Breadboard
- 1* 74HC595
- 8* LED
- 8* Resistor (220Ω)
- Jumper wires

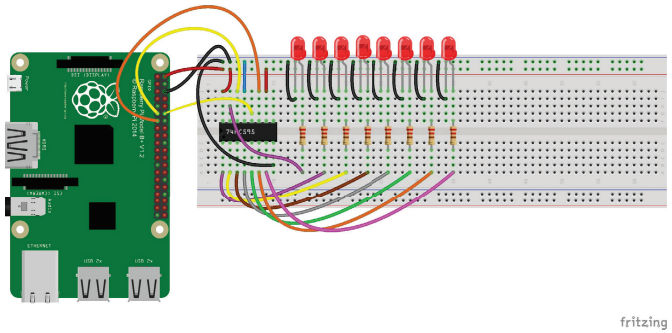
Principle
74HC595 is a silicon CMOS device, which has an 8-bit shift register and a memory with three-state output function. Compatible with low voltage TTL circuit, 74HC595 can transform serial input of 8-bit data into parallel output of 8-bit data. So it is often used to extend GPIO for embedded system and drive low power devices.

In this experiment, we attach ST_CP to Raspberry Pi GPIO1, SH_CP to GPIO2, and DS to GPIO0. Input data in DS pin to the shift register when SH_CP (the clock input of the shift register) is at the rising edge, and to the memory register when ST_CP (the clock input of the memory) is at the rising edge. Then you can control the states of SH_CP and ST_CP via Raspberry Pi GPIO to transform serial input data into parallel output data so as to save Raspberry Pi GPIO's.



Experimental Procedures

Step 1:
Build the circuit



Step 2:
Edit and save the Code in a file called 74HC595_LED.c

Step 3:
Compile

```
gcc 74HC595_LED.c -o 74HC595_LED -lwiringPi
```

Step 4:
Run

```
sudo ./74HC595_LED
```


Here you should see eight LED's blinking regularly, and realistic effect is shown as follow:

Further Exploration
In this experiment, three Raspberry Pi GPIO's are used to separately control 8 LED's based on 74HC595. In fact, 74HC595 has another powerful function – cascade. With cascade, you can use a microprocessor like 3 Raspberry Pi IO's to control more peripherals. This function will be explained in subsequent lessons.

C

30 LCD DISPLAY 1602

Introduction
In this lesson, we will learn how to use LCD1602 to display character strings.

Components
- 1* Breadboard
- 1* LCD1602
- 1* Potentiometer
- Jumper wires

Principle
LCD1602, or character type LCD1602, is a dot matrix LCD module specially used to display letters, figures, symbols, and so on. It consists of many 16*2 dot matrices, and each one is composed of 5*7 or 5*11 character bit. Each character bit can display one character. There is a dot space between each adjacent character bit. Also there is a dot space between each row. The dot space functions as a character space or line space; thus, LCD1602 cannot display graphics very well. It is widely used in pocket instruments and low power application systems due to its micro power consumption, small size, richness in contents, ultra-thinness and lightness.

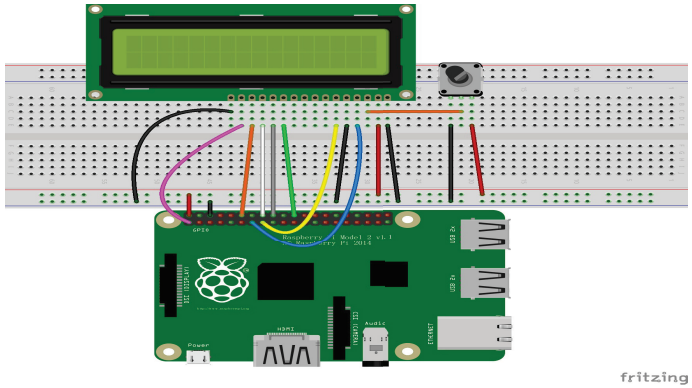
LCD1602 uses the standard 16-pin port, among which:

Pin 1 (GND): connected to Ground;
Pin 2 (Vcc): connected to 5V positive power supply;
Pin 3 (Vo): used to adjust the contrast of LCD1602; the level is lowest when it's connected to positive power supply, and highest when connected to ground (you can connect a 10K potentiometer to adjust its contrast when using LCD1602);
Pin 4 (RS): for register selection; it selects data register when supplied with high level (1), and instruction register when supplied with low level (0);
Pin 5 (R/W): to read/write signals; it reads signals when supplied with high level (1), and writes signals when supplied with low level (0). In this experiment, you only need to write data to LCD1602, so connect this pin to ground;
Pin 6 (E): An enable pin; when supplied with low level, the LCD module will execute relevant instructions;
Pin 7 (D0-D7): pins that read and write data;
A and K: LCD back-light power source.

LCD1602 has two operation modes: 4-bit and 8-bit. When the IO's of microprocessor (MCU) are insufficient, you can choose 4-bit mode, under which only pins D4~D7 are used. After connecting the circuit, you can operate LCD1602 by Raspberry Pi.

Experimental Procedures

Step 1:
Build the circuit



Step 2:
Edit and save the Code in a file called lcd1602_2.c

Step 3:
Compile

```
gcc lcd1602_2.c -o lcd1602_2 -lwiringPiDev -lwiringPi
```

Step 4:
Run

```
sudo ./lcd1602_2
```

You should see two lines of characters displayed on the LCD1602. The realistic effect is as follows:

Further Exploration

In this experiment, LCD1602 is driven in the 4-bit mode. You can do programming by yourself to drive LCD1602 in the 8-bit mode.

31 HOW TO DRIVE A DC MOTOR

Introduction

In this lesson, we will learn to how to use L293D to drive a DC motor and make it rotate clockwise and counterclockwise.

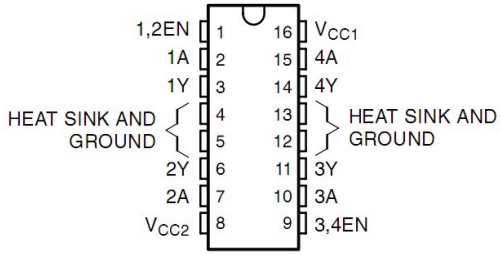
Components

- 1* Breadboard
- 1* L293D
- 1* DC motor
- 1* Power Module
- Jumper wires

Principle

L293D

This is a very practical chip that can independently control two DC motors. In this experiment, just about half of the chip is used – most pins on the right side of the chip are used to control a second motor, which is not needed since we apply only one motor here.

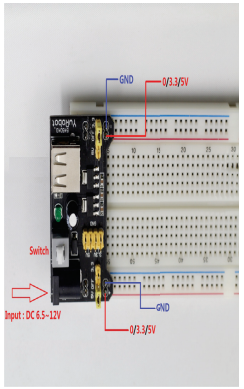


L293D has two pins (Vcc1 and Vcc2) for power supply. Vcc2 is used to supply power for the motor, while Vcc1 to supply for the chip. Since a small-sized DC motor is used here, connect both pins to +5V. If you use a higher power motor, you need to connect Vcc2 to an external power supply. At the same time, the GND of L293D should be connected to that of the board.

DC Motors are devices that turn DC electrical energy into mechanical energy. They are widely used in electrical drive for their superior speed regulation performance.

In this experiment, connect two IO's of the Raspberry Pi to two opto-couplers which isolate the power supply to protect your Raspberry Pi from

interference and crashing when the motor works. Connect the output port of the opto-coupler to the driver IC (L293D) of the motor. When one of the two IO's of the Raspberry Pi is at high level and the other is low, the motor will rotate clockwise or counterclockwise. By adjusting the pulse width, you can realize DC motor speed regulation as well. A motor is an inductive device. Its four diodes play an important role in freewheeling and suppress the surge voltage of motor coils to prevent irreversible damages to driver ICs.



The module comes with a yellow jumper cap, which is for selecting between power off and voltages (3.3V and 5V). You can switch the output voltage (0/3.3/5V) by putting the cap into the exact pin.
In this experiment, it needs large currents to drive the motor especially when it starts and stops, which will severely interfere with the normal work of Raspberry Pi. Therefore, we separately supply power for the motor by this module to make it run safely and steadily.

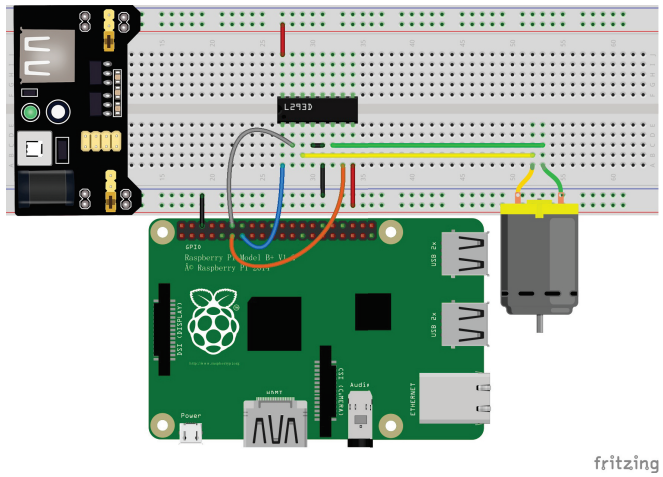
The maximum current allowed for this power supply module is 700mA. You can switch the output voltage (0/3.3/5V) through a yellow jumper cap.

In this experiment, it needs large current to drive the motor especially when it starts and stops, which will severely interfere with the normal work of Raspberry Pi. Therefore, we separately supply power for the motor by this module to make it run safely and steadily.

Experimental Procedures

Step 1:

Build the circuit



Step 2:
Edit and save the Code in a file called motor.c

Step 3:
Compile
gcc motor.c -o motor **-**lwiringPi

Step 4: Run
sudo **./**motor
For Python users:
Step 2: Change directory
cd **/**home**/**pi**/**Sunfounder_SuperKit_Python_code_for_RaspberryPi**/**
Step 3: Run
sudo python 07_motor.py
Now, you should see the motor blade rotating.

Further Exploration
You can use buttons to control the clockwise and counterclockwise rotation of the motor based on the previous contents. Also you can use PWM principle to control the rotation.

Summary

Through this lesson, you have learnt the relative principle and driving mode of DC motors, as well as how to drive a motor by Raspberry Pi. You should also pay special attention to the fact that a DC motor will greatly interfere with the whole circuit when it works, so you need to adopt photoelectric isolation and provide separate power supply. A freewheeling diode is also necessary for the whole system to work reliably and steadily.

LINKS

Raspberry Pi Homepage
http://raspberrypi.org

wiringPi
http://wiringpi.com

Fritzing
http://fritzing.org

Markdown Editor Remarkable
https://remarkableapp.github.io/

GitHub Repository for this Book
https://github.com/eMakerStudio/C_for_Raspberry_Pi

Book Homepage
http://emaker-studio.goprogramming.space/

Proof